

Torch

1. torch.index_select()

1. Example, usage in NMS, a voting algorithm to remove bounding boxes with IoU greater than a threshold t with a bounding box with the biggest confidence score.

```
def bbox_nms(bboxes, scores, threshold=0.5):
    """
    Non maximum suppression
    :param bboxes: (tensor) [N,4]
    :param scores: (tensor) [N,]
    :param threshold: (float)
    :return:
        keep: (tensor) selected indices
    """
    is_torch = False
    if not isinstance(bboxes, np.ndarray):
        bboxes = bboxes.cpu().detach().numpy()
        scores = scores.cpu().detach().numpy()
        is_torch = True
    x1 = bboxes[:, 0]
    y1 = bboxes[:, 1]
    x2 = bboxes[:, 2]
    y2 = bboxes[:, 3]
    areas = (x2 - x1) * (y2 - y1)
    order = scores.argsort()[::-1]

    keep = []
    while len(order) > 0:
        if len(order) == 1:
            i = order.item()
            keep.append(i)
            break
        i = order[0]
        keep.append(i)

        xx1 = x1[order[1:]].clip(min=x1[i])
        yy1 = y1[order[1:]].clip(min=y1[i])
        xx2 = x2[order[1:]].clip(max=x2[i])
        yy2 = y2[order[1:]].clip(max=y2[i])

        w = (xx2 - xx1).clip(min=0)
        h = (yy2 - yy1).clip(min=0)
        inter = w * h

        overlap = inter / (areas[i] + areas[order[1:]] - inter)

        ids = np.where(overlap < threshold)[0]
        if len(ids) == 0:
            break
        order = order[ids + 1]
    return keep if not is_torch else torch.LongTensor(keep)
```

2. torch.gather()

3. torch.tensor.detach().cpu().numpy()

4. device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

5. gradient clipping simple solution to gradient explosion, works well practically :

- torch.nn.utils.clip_grad_norm_(parameters, max_norm, norm_type=2)
- Example: torch.nn.utils.clip_grad_norm_(retinanet.parameters(), 0.1)

6. torch.bmm

1. torch.bmm is a special case of torch.matmul where both the tensors are 3-dimensional and contains equal number of matrices.
2. Example: torch.bmm used in neural style transfer.
3. Code:

```
def gram_matrix(features, normalize=True):
    """
    Compute the Gram matrix from features.

    Inputs:
    - features: PyTorch Variable of shape (N, C, H, W) giving features for
      a batch of N images.
    - normalize: optional, whether to normalize the Gram matrix
      If True, divide the Gram matrix by the number of neurons (H * W * C)

    Returns:
    - gram: PyTorch Variable of shape (N, C, C) giving the
      (optionally normalized) Gram matrices for the N input images.
    """
```

```
#####
# TODO: Implement content loss function                                     #
# Please pay attention to use torch tensor math function to finish it.    #
# Otherwise, you may run into the issues later that dynamic graph is broken #
# and gradient can not be derived.                                         #
#                                                                           #
# HINT: you may find torch.bmm() function is handy when it comes to process #
# matrix product in a batch. Please check the document about how to use it. #
#####
N, C, H, W = features.shape
# (N, C, H, W) to (N, C, H*W)
features = features.view(N, C, -1)
# Create a copy of size (N, H*W, C)
features_copy = features.transpose(2, 1)
# (N, C, H*W) @ (N, H*W, C) = (N, C, C)
gram_matrix = torch.bmm(features, features_copy)
if normalize:
    gram_matrix = gram_matrix / (H*W*C)
return gram_matrix

#####
#                               END OF YOUR CODE                           #
#####
```

7. torch.repeat

1. torch.rand(3).repeat(4, 2) is when array of shape: [1, 3] * (4, 2) -> output shape: [4, 6].
2. Code:

```
a = torch.randn(3, 2)
print(a, a.repeat(4, 2))

a = torch.randn(3)
print(a, a.repeat(4, 2))
print(a, a.repeat(3, 4, 2))
print(a.shape, a.repeat(4, 2).shape)
print(a.shape, a.repeat(3, 4, 2).shape)
if (a.repeat(1, 2) == a.repeat(2)).all():
    print("Yes, a.repeat(1, 2) equals a.repeat(2)")
```

Output:

```
tensor([[ 0.2031,  0.8425],
        [-0.3773,  0.0066],
        [ 0.4972, -0.0822]]) tensor([[ 0.2031,  0.8425,  0.2031,  0.8425],
        [-0.3773,  0.0066, -0.3773,  0.0066],
        [ 0.4972, -0.0822,  0.4972, -0.0822],
        [ 0.2031,  0.8425,  0.2031,  0.8425],
        [-0.3773,  0.0066, -0.3773,  0.0066],
        [ 0.4972, -0.0822,  0.4972, -0.0822],
        [ 0.2031,  0.8425,  0.2031,  0.8425],
        [-0.3773,  0.0066, -0.3773,  0.0066],
        [ 0.4972, -0.0822,  0.4972, -0.0822],
        [ 0.2031,  0.8425,  0.2031,  0.8425],
        [-0.3773,  0.0066, -0.3773,  0.0066],
        [ 0.4972, -0.0822,  0.4972, -0.0822]])
tensor([ 0.7154, -1.4502,  0.6132]) tensor([[ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132]])
tensor([ 0.7154, -1.4502,  0.6132]) tensor([[[ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132]]],
        [[ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132]]],
        [[ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132],
        [ 0.7154, -1.4502,  0.6132,  0.7154, -1.4502,  0.6132]]])
torch.Size([3]) torch.Size([4, 6])
torch.Size([3]) torch.Size([3, 4, 6])
Yes, a.repeat(1, 2) equals a.repeat(2)
```

8. torch.expand

1. Example from Yolov1 loss function, compute IOU function: <https://github.com/Liang-ZX/HKU-DASC7606-A1/blob/main/src/utils/loss.py>

```
def compute_iou(self, box1, box2):
    """ compute IOU between boxes
        - box1 (bs, 4) 4: [x1, y1, x2, y2] left top and right bottom
        - box2 (bs, 4) 4: [x1, y1, x2, y2] left top and right bottom
    """
    N = box1.size(0)
```

```

M = box2.size(0)

lt = torch.max(
    box1[:, :2].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
    box2[:, :2].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
)

rb = torch.min(
    box1[:, 2:].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
    box2[:, 2:].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
)

wh = rb - lt # [N,M,2]
wh[wh < 0] = 0 # clip at 0
inter = wh[:, :, 0] * wh[:, :, 1] # [N,M]

area1 = (box1[:, 2] - box1[:, 0]) * (box1[:, 3] - box1[:, 1]) # [N,]
area2 = (box2[:, 2] - box2[:, 0]) * (box2[:, 3] - box2[:, 1]) # [M,]
area1 = area1.unsqueeze(1).expand_as(inter) # [N,] -> [N,1] -> [N,M]
area2 = area2.unsqueeze(0).expand_as(inter) # [M,] -> [1,M] -> [N,M]

iou = inter / (area1 + area2 - inter)
return iou

```

2. Another example from yolov2: <https://raw.githubusercontent.com/kuangliu/pytorch-yolov2/refs/heads/master/utils.py>

```

def box_iou(box1, box2):
    '''Compute the intersection over union of two set of boxes.

    box1, box2 are as: [xmin, ymin, xmax, ymax].

    Args:
        box1: (tensor) bounding boxes, sized [N,4].
        box2: (tensor) bounding boxes, sized [M,4].

    Return:
        (tensor) iou, sized [N,M].
    '''
    N = box1.size(0)
    M = box2.size(0)

    lt = torch.max(
        box1[:, :2].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
        box2[:, :2].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
    )

    rb = torch.min(
        box1[:, 2:].unsqueeze(1).expand(N, M, 2), # [N,2] -> [N,1,2] -> [N,M,2]
        box2[:, 2:].unsqueeze(0).expand(N, M, 2), # [M,2] -> [1,M,2] -> [N,M,2]
    )

    wh = (rb - lt).clamp(min=0) # [N,M,2]
    inter = wh[:, :, 0] * wh[:, :, 1] # [N,M]

    area1 = (box1[:, 2] - box1[:, 0]) * (box1[:, 3] - box1[:, 1]) # [N,]
    area2 = (box2[:, 2] - box2[:, 0]) * (box2[:, 3] - box2[:, 1]) # [M,]
    area1 = area1.unsqueeze(1).expand_as(inter) # [N,] -> [N,1] -> [N,M]
    area2 = area2.unsqueeze(0).expand_as(inter) # [M,] -> [1,M] -> [N,M]

    iou = inter / (area1 + area2 - inter)
    return iou

```

3. Simpler example:

Code:

```

x = torch.rand(1, 3)
N = 5
x = x.expand(N, 3)
print(x)

```

Output:

```

tensor([[0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364],
        [0.3884, 0.0457, 0.4364]])

```

9. torch.nn.Embedding VS torch.nn.Linear

1. torch.nn.Embedding is a lookup table. it works the same as torch.tensor but with a few twists (like possibility of using sparse embedding or default value at specified index).

Code: p8.py

```

import torch

# nn.Embedding

```

```

N, T, V, D = 2, 3, 4, 5
embedding = torch.nn.Embedding(V, D)
caption = torch.randint(0, V, (N, T))
x = embedding(caption)
print(x.shape)
print(N, T, D)
print(x)

```

Output:

```

torch.Size([2, 3, 5])
2 3 5
tensor([[[[-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077]],

        [[[-0.4294, -0.3665, -0.1000, -1.1245,  0.3728],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077],
          [-0.3991, -0.5553,  0.6418, -1.2668, -0.3077]]]])
grad_fn=<EmbeddingBackward0>)

```

2. `torch.nn.Linear` **only modifies the last dimension of the input**. `nn.Linear` without bias is a `torch.Tensor` (weight) but it does operation on the input and the weight, which is essentially `output = input.matmul(weight, t())`:

Code: p8.py

```

# nn.Linear
N, D1, D2 = 2, 3, 4
x1 = torch.rand(N, D1)
linear = torch.nn.Linear(D1, D2)
x2 = linear(x1)
print(x2.shape)
print(N, D2)
print(x2)

```

Output:

```

torch.Size([2, 4])
2 4
tensor([[[-0.0952, -0.2378,  0.1940,  0.4159],
          [-0.4102,  0.2701,  0.7802,  0.5189]], grad_fn=<AddmmBackward0>])

```

10. `torch.Tensor.unsqueeze`

Code:

```

a = torch.rand(5, 1)
b = torch.rand(3)
# Using * for outer product
c = a * b
print(c)

d = a @ b.unsqueeze(0)
print(d)

print((c == d).all())

```

Output:

```

tensor([[[0.5483, 0.3419, 0.4980],
          [0.3854, 0.2403, 0.3500],
          [0.7277, 0.4538, 0.6610],
          [0.2675, 0.1668, 0.2430],
          [0.4835, 0.3015, 0.4391]]])
tensor([[[0.5483, 0.3419, 0.4980],
          [0.3854, 0.2403, 0.3500],
          [0.7277, 0.4538, 0.6610],
          [0.2675, 0.1668, 0.2430],
          [0.4835, 0.3015, 0.4391]]])
tensor(True)

```

11. `torch.Tensor.unsqueeze(1)` VS `x[:, None]`

1. `[None, ...]` is numpy notation for `.unsqueeze(0)`. The two are equivalent. The version with advanced indexing might be slower because it has more checking to do to find out exactly what you want to do.

Code:

```

x = torch.rand(3, 4, 5)
print(type(x))
a = x.unsqueeze(1)
b = x[:, None]
print((a==b).all())

```

Output:

```

<class 'torch.Tensor'>
tensor(True)

```

12. `torch.from_numpy` vs `torch.tensor`

Slicing and Indexing in torch

1. if x is of shape (N, D) , how priority of indexing works: dim N first, then dim D .
2. Example: $x[0]$ will return the second row of the array x , or equivalent to $x[0, :]$.

Code:

```
N = 3
T = 5
D = 10
x = torch.rand(N, T, D)
print(x[1].shape)
print(x[1, :, :].shape)
print((x[1] == x[1, :, :]).all())
```

Output:

```
torch.Size([5, 10])
torch.Size([5, 10])
tensor(True)
```

3. Code: pytorch_slicing_and_indexing.py. See: yolov2 box encoder in [Object_Detection](#)

```
# -*- coding: utf-8 -*-
"""
Created on Mon Jun 15 19:27:03 2026

@author: reina
"""

import torch

"""
Tensor slicing and indexing example,
taken from yolov2 box encoder code.
Context is not important here.
"""

if __name__ == '__main__':
    num_anchors_per_grid_cell = 5 # no. of anchor variations
    # num_anchors(total) = fmsize*fmsize*num_anchors_per_grid_cell
    fmsize = 10
    num_boxes = 3 # no. of gt bboxes
    ious = torch.randn(fmsize, fmsize, num_anchors_per_grid_cell, num_boxes) # [10, 10, 5, 3]
    print("ious.shape", ious.shape) # shape: [fmsize, fmsize, num_anchors_per_grid_cell, num_boxes]
    loc_targets = torch.zeros(num_anchors_per_grid_cell, 4, fmsize, fmsize) # shape: [num_anchors_per_grid_cell,
4coords, fmsize, fmsize] = [5, 4, 10, 10]
    cls_targets = torch.zeros(num_anchors_per_grid_cell, 20, fmsize, fmsize)
    # for i in range(num_boxes):
    i = 2
    cx, cy = 4, 3
    _, max_idx = ious[cy, cx, :, i].max(0, keepdim=True)
    print("max_idx", max_idx)
    print("ious[cy, cx, :, i].shape", ious[cy, cx, :, i].shape) # shape: [num_anchors]
    j = max_idx[0]
    print("loc_targets[j, :, cy, cx].shape", loc_targets[j, :, cy, cx].shape) # shape: [4coords]
    loc_targets[j, :, cy, cx] = torch.rand(4)
```

Output:

```
ious.shape torch.Size([10, 10, 5, 3])
max_idx tensor([4])
ious[cy, cx, :, i].shape torch.Size([5])
loc_targets[j, :, cy, cx].shape torch.Size([4])
```

Helper functions for images

```
def preprocess(img, size=512):
    transform = T.Compose([
        T.Resize(size),
        T.ToTensor(),
        T.Normalize(mean=SQUEEZENET_MEAN.tolist(),
                    std=SQUEEZENET_STD.tolist()),
        T.Lambda(lambda x: x[None]),
    ])
    return transform(img)

def deprocess(img):
    transform = T.Compose([
        T.Lambda(lambda x: x[0]),
        T.Normalize(mean=[0, 0, 0], std=[1.0 / s for s in SQUEEZENET_STD.tolist()]),
        T.Normalize(mean=[-m for m in SQUEEZENET_MEAN.tolist()], std=[1, 1, 1]),
        T.Lambda(rescale),
        T.ToPILImage(),
    ])
    return transform(img)

def rescale(x):
```

```

low, high = x.min(), x.max()
x_rescaled = (x - low) / (high - low)
return x_rescaled

def rel_error(x,y):
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

def features_from_img(imgpath, imgsize):
    img = preprocess(PIL.Image.open(imgpath), size=imgsize)
    img_var = img.type(dtype)
    return extract_features(img_var, cnn), img_var

```

Extracting features

1. Using `cnn_model._modules.values()`

```

# We provide this helper code which takes an image, a model (cnn), and returns a list of
# feature maps, one per layer.
def extract_features(x, cnn):
    """
    Use the CNN to extract features from the input image x.

    Inputs:
    - x: A PyTorch Variable of shape (N, C, H, W) holding a minibatch of images that
      will be fed to the CNN.
    - cnn: A PyTorch model that we will use to extract features.

    Returns:
    - features: A list of feature for the input images x extracted using the cnn model.
      features[i] is a PyTorch Variable of shape (N, C_i, H_i, W_i); recall that features
      from different layers of the network may have different numbers of channels (C_i) and
      spatial dimensions (H_i, W_i).
    """
    features = []
    prev_feat = x
    for i, module in enumerate(cnn._modules.values()):
        next_feat = module(prev_feat)
        features.append(next_feat)
        prev_feat = next_feat
    return features

# -*- coding: utf-8 -*-
"""
Created on Tue Mar 31 15:08:13 2026

@author: reina
"""
import torch
import torchvision
from pprint import pprint

dtype = torch.float32
model = torchvision.models.resnet50()
cnn = torchvision.models.squeezenet1_1(pretrained=True)
features = cnn.features # Layers in cnn
print(features.type(dtype))

feats = []
N = 1; C=3; H=224; W=224 # properties of an image
prev_feat = torch.rand(N, C, H, W)
for i, module in enumerate(cnn._modules.values()):
    next_feat = module(prev_feat)
    feats.append(next_feat)
    prev_feat = next_feat
print(len(features))
print(feats[0].shape)

print(list(enumerate(cnn.parameters()))) # weights

l = [(n, v.shape) for n, v in list(cnn.named_parameters())]
pprint(l, indent=2)

```

2. Note:

1. Layers of a net (inherited from `nn.Module`) are stored in `Module._modules`, which is initialized in `__construct`.
2. `self._modules` is updated in `__setattr__`. `__setattr__(obj, name, value)` is called when `obj.name = value` is executed.
3. If one runs a forward pass of a net inherited from `nn.Module`, the `nn.Module.__call__` will be called, in which `self.forward` is called. However, one has overrode the `forward` when implementing the net.

Pytorch Loss-Input Confusion

1. `torch.nn.functional.binary_cross_entropy` takes logistic sigmoid `nn.Sigmoid()` values as inputs.
2. `torch.nn.functional.binary_cross_entropy_with_logits` takes logits as inputs.

3. `torch.nn.functional.cross_entropy` takes logits as inputs (performs `log_softmax` internally, no `Softmax` needed just `nn.Linear` as final layer, `Softmax` is built-in into the loss function!)
4. `torch.nn.functional.nll_loss` is like `cross_entropy` but takes log-probabilities (log-softmax) `nn.Softmax(dim=1)` or `nn.LogSoftmax(dim=1)` values as inputs.

Tensor shapes



- 1.
2. in a torch tensor there is usually a dimension that is the most important and can be regarded as the 'meat' or 'heart' of the tensor. If we can imagine a tensor as a fruit, then the 'meat' is the channel (C) dimension, and other dimensions are the skin of the fruit, they are usually either the batch size (N), spatial (H, W) or temporal (T) dimensions. For example,
 1. in an image is (H, W, 3) the heart is the C=3 dimension.
 2. in a collection of points, (N, D) the heart is the features D dimension.
 3. In a yolo head output tensor (N, B*5 + C, H, W) the heart is in the second dimension, which are the things we're trying to predict, B is the number of prediction boxes, 5 being the box coordinates `xyxy + 1` confidence score, and C being the number of classes.
 4. In video data: (H, W, 3, F) → (HW/p**2, 3p**2, F), again the second dimension is the features dimension, with F being the number of RGB frames (temporal) and HW/p**2 being the number of tokens (source: timesformer).
3. From Simple BEV repo: We maintain consistent axis ordering across all tensors. In general, the ordering is N, T, C, Z, Y, X, where
 - N: batch
 - T: Sequence (for temporal or multiview data)
 - C: channels, or often also written as D
 - Z: depth
 - Y: height
 - X: width

The ordering stands even if a tensor is missing some dims, for example, plain images are B, C, Y, X (as is the pytorch standard).

Axis directions

- Z: forward
- Y: down
- X: right

This means the top-left of an image is "0.0", and coordinates increase as you travel right and down. z increases forward because it's the depth axis.

<code>nn.Sequential</code>	<code>nn.ModuleList</code>

<code>model.named_children()</code>	<code>model.named_modules()</code>

<code>detach()</code>	<code>requires_grad(False)</code>
detaches a tensor from the computation graph, it creates a new tensor that shares the same data. However the new tensor is no longer being tracked by autograd.	temporarily disables gradient tracking by setting <code>requires_grad</code> flag to false.

Data Augmentation

1. Code: `data_preprocessing_img.py`

```

# -*- coding: utf-8 -*-
"""
Created on Wed Jun  3 17:57:10 2026

@author: reina
"""

import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

import os

# !mkdir lorem_data
# !mkdir lorem_data/lorem_data1
# !mkdir lorem_data/lorem_data2
# !mkdir lorem_data/lorem_data3
# !curl -o lorem_data/lorem_data1/sample1.jpg https://fastly.picsum.photos/id/22/4434/3729.jpg?
hmac=fjZdkSMZJNFgsoDh8Qo5zdA_nSGUAWvKLyqmEt2xs0
# !curl -o lorem_data/lorem_data2/sample2.jpg https://fastly.picsum.photos/id/16/2500/1667.jpg?
hmac=uAkZwYc5phCRNFTrV_prJ_0rP0EdwJaZ4ctje2bY7aE
# !curl -o lorem_data/lorem_data3/sample3.jpg https://fastly.picsum.photos/id/64/4326/2884.jpg?
hmac=9_SzX666YRpR_f0yYStXpfSiJ_ed03ghlSRnH2w09Kg

# Function to resize an image
def resize_image(input_path, output_path, size):
    try:
        img = Image.open(input_path)
        img = img.resize(size, Image.Resampling.LANCZOS)
        img.save(output_path)
        print(f"Resized {input_path} to {output_path}")
    except Exception as e:
        print(f"Error resizing {input_path}: {e}")

# Here is a helper function to calculate the mean and standard deviation of the images
def get_mean_and_std(dataloader):
    channels_sum, channels_squared_sum, num_batches = 0, 0, 0

    for data, _ in dataloader:
        # Sum over all batches, height, and width; keep the channel dimension
        channels_sum += torch.sum(data, dim=[0, 2, 3])
        channels_squared_sum += torch.sum(data ** 2, dim=[0, 2, 3])
        num_batches += data.shape[0] # Add the batch size

    mean = channels_sum / (num_batches * dataloader.dataset[0][0].size(1) * dataloader.dataset[0][0].size(2))

    std = (channels_squared_sum / (num_batches * dataloader.dataset[0][0].size(1) * dataloader.dataset[0][0].size(2))
    - mean ** 2) ** 0.5

    return mean, std

if __name__ == '__main__':
    # Define the directories containing the images
    directories = [
        "lorem_data/lorem_data1",
        "lorem_data/lorem_data2",
        "lorem_data/lorem_data3",
    ]

    # Size for resizing
    new_size = (224, 224)

    for directory in directories:
        for filename in os.listdir(directory):
            if filename.endswith(".jpg"):
                input_path = os.path.join(directory, filename)
                output_path = os.path.join(directory, filename)
                resize_image(input_path, output_path, new_size)

    DATA_DIR = 'lorem_data/'

    ## Pytorch simplifies the creation of image datasets
    dataset = torchvision.datasets.ImageFolder(DATA_DIR, transform=torchvision.transforms.ToTensor()) # We apply a
    transform toTensor which converts the images to pytorch tensors

    ## A snapshot of the data without any transformation or normalization
    r, c = [1, 3]
    fig, ax = plt.subplots(r, c, figsize=(15, 5)) \

    k = 0
    dtl_pure = torch.utils.data.DataLoader(
        dataset=dataset,
        batch_size=3,)

    for data in dtl_pure:

```

```

x, y = data
for j in range(c):
    img = x[k].numpy().transpose(1, 2, 0)
    ax[j].imshow(img) # Adjusted indexing for one-dimensional ax
    ax[j].axis('off')
    k += 1
    if k >= 3: # Break the loop after 3 images
        break
break # This will only execute the outer loop once

# Define normalization transforms:
data_loader = torch.utils.data.DataLoader(dataset=dataset, batch_size=3)
mean, std = get_mean_and_std(data_loader)
print("Means: ", mean)
print("std_dev: ", std)
normalize_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=mean, std=std),])
r, c = [2, 3]
fig, ax = plt.subplots(r, c, figsize=(15, 10))
# Apply normalization to both training and test datasets:
dtl_norm = torch.utils.data.DataLoader(
    dataset=torchvision.datasets.ImageFolder(DATA_DIR, transform=normalize_transform),
    batch_size=3,)

# Display images from dtl_pure
for data in dtl_pure:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[0, j].imshow(img)
        ax[0, j].axis('off')
    break # Only take the first batch

# Display images from dtl_norm
for data in dtl_norm:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[1, j].imshow(img)
        ax[1, j].axis('off')
    break # Only take the first batch

# Data augmentation
# Apply data augmentation to the training dataset:

horiz_flip_transform = torchvision.transforms.Compose([
    # transforms.Resize((224, 224)), # Ensures that the images are resized to 224x224
    # transforms.RandomHorizontalFlip(p=1), # Flips the image horizontally randomly with a
probability of 100%
    # transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5, hue=0.2), # Randomly
changes the brightness, contrast, saturation, and hue
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1)), # Random affine
transformation of the image keeping center invariant
    transforms.ToTensor(), # Converts the image to a PyTorch tensor
])

r, c = [2, 3]
fig, ax = plt.subplots(r, c, figsize=(15, 10))
# Apply normalization to both training and test datasets:
dtl_hori_flip = torch.utils.data.DataLoader(
    dataset=torchvision.datasets.ImageFolder(DATA_DIR, transform=horiz_flip_transform),
    batch_size=3,)

# Display images from dtl_pure
for data in dtl_pure:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[0, j].imshow(img)
        ax[0, j].axis('off')
    break # Only take the first batch

# Display images from dtl_norm
for data in dtl_hori_flip:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[1, j].imshow(img)
        ax[1, j].axis('off')
    break # Only take the first batch

# Multiple data augmentation can be combined
# Apply data augmentation:

combine_transform = torchvision.transforms.Compose([
    transforms.Resize((224, 224)), # Ensures that the images are resized to 224x224
    transforms.RandomHorizontalFlip(p=0.5), # Flips the image horizontally randomly with a
probability of 50%
    transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5, hue=0.2), # Randomly

```

```

changes the brightness, contrast, saturation, and hue
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1)), # Random affine
transformation of the image keeping center invariant
    transforms.ToTensor(), # Converts the image to a PyTorch tensor
    transforms.Normalize(mean=mean, std=std)
])

r, c = [2, 3]
fig, ax = plt.subplots(r, c, figsize=(15, 10))
# Apply combination of transforms :
combined_dtl = torch.utils.data.DataLoader(
    dataset=torchvision.datasets.ImageFolder(DATA_DIR, transform=combine_transform),
    batch_size=3,)

# Display images from dtl_pure
for data in dtl_pure:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[0, j].imshow(img)
        ax[0, j].axis('off')
    break # Only take the first batch

# Display images from dtl_norm
for data in combined_dtl:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[1, j].imshow(img)
        ax[1, j].axis('off')
    break # Only take the first batch

# Apply data augmentation to the training dataset:

vert_flip_transform = torchvision.transforms.Compose([
    transforms.Resize((224, 224)), # Ensures that the images are resized to 224x224
    transforms.RandomVerticalFlip(p=1), # Flips the image horizontally randomly with a probability
of 100%
    transforms.ColorJitter(brightness=0.5, contrast=0.5, saturation=0.5, hue=0.2), # Randomly
changes the brightness, contrast, saturation, and hue
    transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1)), # Random affine
transformation of the image keeping center invariant
    transforms.ToTensor(), # Converts the image to a PyTorch tensor
])

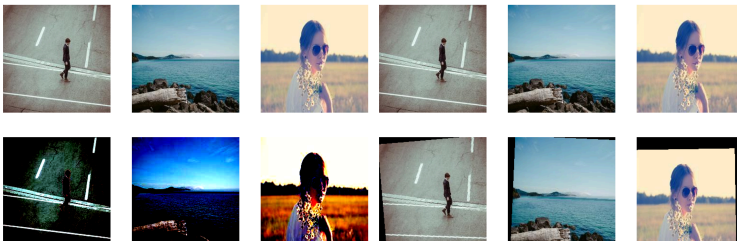
r, c = [2, 3]
fig, ax = plt.subplots(r, c, figsize=(15, 10))
# Apply vertical flipping augmentation:
dtl_vert_flip = torch.utils.data.DataLoader(
    dataset=torchvision.datasets.ImageFolder(DATA_DIR, transform=vert_flip_transform),
    batch_size=3,)

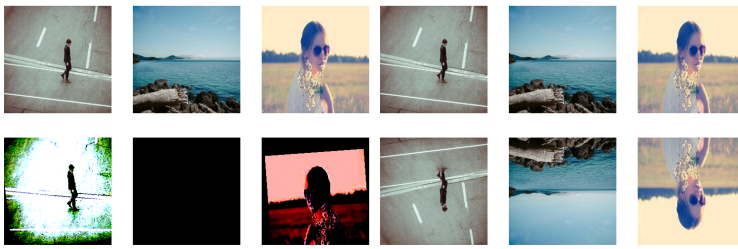
# Display images from dtl_pure
for data in dtl_pure:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[0, j].imshow(img)
        ax[0, j].axis('off')
    break # Only take the first batch

# Display images from dtl_vert_flip
for data in dtl_vert_flip:
    x, y = data
    for j in range(c):
        img = x[j].numpy().transpose(1, 2, 0)
        ax[1, j].imshow(img)
        ax[1, j].axis('off')
    break # Only take the first batch

```

2. Plots:





3. Code: data_preprocessing.py

```
# -*- coding: utf-8 -*-
"""
Created on Wed Jun 3 16:05:28 2026

@author: reina
"""

import cv2
import random
import numpy as np

color_red = (0, 0, 255)

def click_event(event, x, y, flags, params):
    if event == cv2.EVENT_LBUTTONDOWN:
        print(x, " ", y)
        cv2.circle(img, (x, y), 1, color_red, 2)
        cv2.imshow(fname, img)

def randomScale(img, bbox):
    # Create copies
    img_copy = img.copy()
    bbox_copy = bbox.copy()
    h, w, _ = img.shape

    # Resize image
    scale = random.uniform(0.8, 1.2)
    img_copy = cv2.resize(img_copy, (int(w * scale), h)) # bgr
    scale_tensor = np.array([scale, 1, scale, 1]) # We only modified w
    bbox_copy = bbox_copy * scale_tensor
    return img_copy, bbox_copy

def random_flip(img, bbox):
    # Create copies
    img_copy = img.copy()
    bbox_copy = bbox.copy()

    img_lr = np.fliplr(img_copy).copy()
    h, w, _ = img_copy.shape
    xmin = w - bbox_copy[2]
    xmax = w - bbox_copy[0]
    bbox_copy[0] = xmin
    bbox_copy[2] = xmax
    return img_lr, bbox_copy.astype(int)

if __name__ == "__main__":
    fname = "n01532829_1261.JPEG"
    img = cv2.imread(fname)
    img_copy = img.copy()
    h, w, _ = img.shape
    is_running = True
    while is_running:
        cv2.imshow(fname, img)
        cv2.setMouseCallback(fname, click_event)
        keyPressed = cv2.waitKey(0)
        if keyPressed == ord("q"):
            is_running = False
        if keyPressed == ord("c"):
            img = img_copy.copy()
            cv2.destroyAllWindows()

    # xyxy
    bbox = np.array([47, 132, 390, 367])
    # bbox_copy = bbox.copy()
    # Plot
    cv2.rectangle(img, (bbox[0], bbox[1]), (bbox[2], bbox[3]), (0, 0, 255), 1)
    cv2.imshow(fname, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    img = img_copy.copy()
    # Apply random scaling
    img_scaled, bbox_scaled = randomScale(img, bbox)
```

```

# Normalize bbox
new_h, new_w, _ = img_scaled.shape
bbox_normalized = bbox_scaled / np.array([new_w, new_h, new_w, new_h])
# Decode bbox
bbox_decoded = (bbox_normalized * np.array([new_w, new_h, new_w, new_h])).astype(
    int
)

print(bbox)
print(bbox_normalized)
print(bbox_decoded)

# Plot
cv2.rectangle(
    img_scaled,
    (bbox_decoded[0], bbox_decoded[1]),
    (bbox_decoded[2], bbox_decoded[3]),
    (0, 0, 255),
    1,
)
cv2.imshow(fname + "_scaled", img_scaled)
cv2.imwrite(fname + "_scaled" + ".png", img_scaled)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Apply random flip
# img = img_copy.copy()
# bbox = bbox_copy.copy()
img_flipped, bbox_flipped = random_flip(img, bbox)
print("bbox_flipped", bbox_flipped)
# Plot
cv2.rectangle(
    img_flipped,
    (bbox_flipped[0], bbox_flipped[1]),
    (bbox_flipped[2], bbox_flipped[3]),
    (0, 0, 255),
    1,
)
cv2.imshow(fname + "_flipped", img_flipped)
cv2.imwrite(fname + "_flipped" + ".png", img_flipped)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

4. Plots:

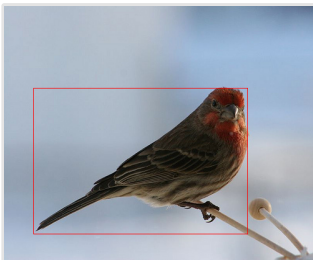
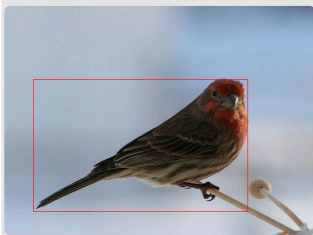
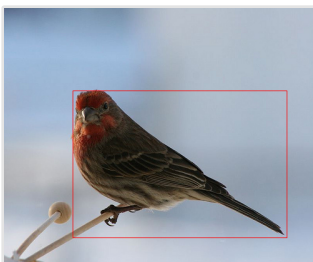


Image with bbox of a bird



Scaled image



Flipped image

image are from minilImageNet

pytorch-book

1. Chapter 2

```
1. import torch
   device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

   net.to(device)
   images = images.to(device)
   labels = labels.to(device)
   output = net(images)
   loss = criterion(output, labels)
```

Pytorch

Tensor

1. From an interface perspective, operations on Tensors can be divided into two categories:
 1. torch.function, such as torch.save, etc.
 2. tensor.function, such as tensor.view, etc.
2. For ease of use, most operations on Tensors support both types of interfaces simultaneously, and this book does not make a specific distinction, for example, torch.sum(a, b) and a.sum(b) are functionally equivalent.
3. From a storage perspective, operations on Tensors can be divided into two categories:
 1. A function that does not modify its own data, such as a.add(b) will return a new Tensor as the result of the addition.
 2. A function that modifies its own data, such as a.add_(b) will still store the result if the addition in a, but a will be modified.
4. Functions whose names end with an underscore are inplace functions, meaning they modify the caller's own data. This distinction is important in practical applications.

Creating Tensors

1. There are many ways to create tensors in Pytorch, as shown in Table 3-1.

Function	Functionality
Tensor(*sizes)	Basic constructor
tensor(data,)	Constructor similar to np.array()
ones(*sizes)	Tensor with all ones
zeros(*sizes)	Tensor with all zeros
eye(*sizes)	The diagonal is 1, and the rest are 0
arange(s, e, step)	From s to e, the step size is step
linspace(s, e, steps)	From s to e, divide evenly into steps
rand/randn(*sizes)	uniform/standard distribution
normal(means, std)/uniform(from, to)	normal distribution/ uniform distribution
randperm(m)	random arrangement
tensor.new_*/torch.*_like	create a Tensor of the same size, filled with * type (e.g. tensor_new_ones, is filled with all ones), and with the same torch.dtype and torch.device

2. All these creation methods allow you to specify the data type (dtype) and the storage device (CPU/GPU) during creation.
3. The tensor function can be used to create a new Tensor. It can accept a list and create a new tensor based on the data in the list. It can also create a tensor based on the data in the list. It can also create a tensor based on a specified data shape and can pass in other tensors. Several examples will be given below.
4. It's important to note that when t.Tensor(*sizes) creates a Tensor, the system doesn't immediately allocate space. It only calculates whether there's enough remaining memory and allocates space immediately after tensor creation. In practice we recommend using torch.tensor instead of torch.Tensor. Let's compare the differences between the two.
5. torch.Tensor is a python class, and by default it is torch.FloatTensor(). Running torch.Tensor([2, 3]) will directly call the __init__() constructor of the Tensor class, generating a single-precision floating-point Tensor(FloatTensor).
6. torch.tensor is a python function with the prototype torch.tensor(data, dtype=None, device=None, requires_grad=False). data supports types such as list, tuple, array, and scalar. torch.tensor() directly copies data from data and generates the

corresponding Tensor based on the original data type.

7. Since `torch.tensor()` can generate a corresponding Tensor based on the data type, we recommend using the `torch.tensor()` method to create a new Tensor in practical applications.

Types of Tensor

8. Tensor types can be further divided into device type and data type (dtype). Device type is divided into CUDA and CPU types, and numeric types include bool, float, and int.
9. Tensor data types are shown in Table 3-2, and each data type has CPU and GPU versions. Readers can obtain the device type of a tensor using `tensor.device` and the data type using `tensor.dtype`.
10. The default data type of a Tensor is `FloatTensor`. Readers can modify the default Tensor type using `torch.set_default_tensor_type` (if the default type is a GPPU Tensor, then all operations will be performed on the GPU) or `torch.set_default_dtype` (only accepts float input types).
11. Understanding the type of a Tensor is helpful for analyzing memory usage. For example, a `FloatTensor` of shape (1000, 1000, 1000) has $1000 \times 1000 \times 1000 = 10^9$ elements, each occupy $32\text{bit} \div 8 = 4\text{byte}$ of memory/GPU memory. `HalfTensor` is specifically designed for GPU versions. With the same number of elements, `HalfTensor` uses only half the GPU memory of a `FloatTensor`.
12. Therefore, using `HalfTensor` can greatly alleviate the problem of insufficient GPU memory. It should be noted that `HalfTensor` has limited numerical values and precision, which may lead to data overflow issues.
13. Tensor data type

Data Type	dtype	CPU tensor	GPU tensor
32-bit floating-point	<code>torch.float32</code> or <code>torch.float</code>	<code>torch.FloatTensor</code>	<code>torch.cuda.FloatTensor</code>
64-bit floating-point	<code>torch.float64</code> or <code>torch.double</code>	<code>torch.DoubleTensor</code>	<code>torch.cuda.DoubleTensor</code>
16-bit half-precision floating-point	<code>torch.float16</code> or <code>torch.half</code>	<code>torch.HalfTensor</code>	<code>torch.cuda.HalfTensor</code>
8-bit unsigned integer	<code>torch.uint8</code>	<code>torch.ByteTensor</code>	<code>torch.cuda.ByteTensor</code>
8-bit signed integer	<code>torch.int8</code>	<code>torch.CharTensor</code>	<code>torch.cuda.CharTensor</code>
16-bit signed integer	<code>torch.int16</code> or <code>torch.short</code>	<code>torch.ShortTensor</code>	<code>torch.cuda.ShortTensor</code>
32-bit signed integer	<code>torch.int32</code> or <code>torch.int</code>	<code>torch.IntTensor</code>	<code>torch.cuda.IntTensor</code>
64-bit signed integer	<code>torch.int64</code> or <code>torch.long</code>	<code>torch.LongTensor</code>	<code>torch.cuda.LongTensor</code>
Boolean type	<code>torch.bool</code>	<code>torch.BoolTensor</code>	<code>torch.cuda.BoolTensor</code>

7. Common methods for converting between different types of tensors are as follows:

1. The most common approach is `tensor.type(new_type)`, but there are also shortcut methods such as `tensor.float()`, `torch.long()`, and `tensor.half()`.
2. Conversion between CPU tensors and GPU tensors is achieved using `tensor.cuda()` and `tensor.cpu()`, and `tensor.to(device)` can also be used.
3. Creating tensors of the same type: `torch.*_like` and `torch.new_*`. These two methods are suitable for writing device compatible code:
 1. `torch.*_like(tensorA)` generates a new tensor with the same attributes as `tensorA` (such as type, shape and CPU/GPU)
 2. `tensor.new_*(new_shape)` creates a new tensor with a different shape but the same attributes.

References:

1. Pytorch book
2. Modern Computer Vision with PyTorch: Explore deep learning concepts and implement over 50 real-world image applications, Kishore Ayyadevara.
3. [How pytorch's nn.Module register submodule, stackoverflow](#)
4. [NMS implementation](#).